

WSServiceWS — revisão geral

pós-fechamento de escopo

Levantamento de dívida técnica e criticidades no código de produção, sem nenhuma refatoração aplicada. Serve de mapa para priorizar o próximo ciclo, começando pela criação de uma **TestDataFactory** centralizada.

211 classes Apex 66 classes de teste 5 factories de teste dispersas

Branch `feat/claude/area-participante-existing-fix`

01 Sumário executivo

Duas dívidas dominam o quadro: (1) ausência de uma TestDataFactory central, que hoje custa boilerplate repetido em dezenas de classes de teste toda vez que uma Validation Rule ou campo obrigatório muda; e (2)

`GestaoSLAService.cls` / `GestaoSLASelector.cls` concentrando 4 domínios de negócio distintos numa única classe cada, mais um caso de funcionalidade (atribuição automática de Entitlement) implementada e testada mas nunca conectada ao fluxo real de Case. Nenhum dos achados críticos é um bug ativo já quebrando produção hoje — são riscos de manutenibilidade e um gap de wiring que tende a virar bug quando alguém assumir, por engano, que a funcionalidade está ativa. Os achados de segurança (FLS/CRUD) são reais, mas mitigados por contexto de uso.

2

ACHADOS CRÍTICOS

6

ACHADOS ALTOS

6

ACHADOS MÉDIOS

3ACHADOS BAIXOS /
REFERÊNCIA

02 Dívida técnica: ausência de TestDataFactory central

Confirmado por grep em todas as 66 classes de teste. As 5 factories existentes não cobrem os SObjects mais recriados manualmente, e nenhuma cria

Account / Contact — a base de praticamente todo teste do domínio de atendimento.

O que as 5 factories atuais realmente fazem

FACTORY	COBRE	PROBLEMA
WS_EmailToCaseTestDataFactory	Case via Email-to-Case, resolve RecordType/Status dinamicamente	A mais genérica das 5 — mas es só ao próprio domínio, não reaproveitável
AreaParticipanteTestDataFactory	AreaParticipante__c, GestaoSLA__c, RegrasSLACategorizacao__c	Helpers de GestaoSLA/RegraSLA, private — não reaproveitáveis em outras suites
CategorizacaoTestDataFactory	Categorizacao__c	Lógica diverge de CaseCreationTestDataFactory, CaseRecategorizationTestDataFactory — 3 versões diferentes do mesmo conceito
CaseCreationTestDataFactory	Case + Categorizacao__c	Regra de Rebocadores (Navio/Procedimentos obrigatórios) codificada de novo
CaseRecategorizationTestDataFactory	Case + Categorizacao__c	Mesma regra de Rebocadores codificada uma terceira vez

Duplicação real, contada via grep (de 66 classes de teste)

PADRÃO	CLASSES QUE DUPLICAM	LEITURA
new Account()	19	Maior gap — nenhuma factory cria Account
new Case()	19	Cada suite monta o Case do zero
new GestaoSLA__c()	15	Helper existe mas é privado numa única factory

PADRÃO	CLASSES QUE DUPLICAM	LEITURA
<code>new Contact()</code>	13	Sem nenhuma factory hoje
<code>new RegrasSLACategorizacao__c()</code>	13	Idem — helper privado
<code>new Group()</code> (filas)	12	Sem factory
<code>new AreaParticipante__c()</code>	10	Factory existe, pouco reaproveitada
<code>new User()</code>	8	Sem factory
<code>new Categorizacao__c()</code> isolado	4	3 implementações divergentes concorrendo
<code>MarcoSLA__c</code> / <code>Reclamacao__c</code> / <code>Agendamento__c</code> / CMDT	0	Sem cobertura de factory nenhuma (gaps reais, não duplicação)

Por que isso é dívida técnica de verdade, não só estilo: 5 Validation Rules ativas no Case hoje forçam obrigatoriedade condicional — nenhum campo é `required` declarativo no schema. Duas delas (`Bloqueio_Navio_Nao_Informado` , `Bloqueio_Porto_Nao_Informado`) exigem `NavioNome__c` / `PortoNome__c` sempre que o `RecordType` é `Rebocadores`, e essa regra está hardcoded em pelo menos 3 lugares diferentes de setup de teste. Qualquer nova Validation Rule no Case (padrão recorrente neste projeto — 5 pacotes recentes criaram VRs novas) exige tocar em cada uma dessas classes de teste manualmente, em vez de um único ponto.

Inconsistências concretas (setup manual quando já existia helper)

- `CaseOpeningNotificationServiceTest.cls:104-118` — recria Account→Contact→Case do zero
- `CaseSurveyDispatchServiceTest.cls:173-196` — idem
- `WWillCaseCreationTest.cls:88-97,132-139` — idem, duas vezes na mesma classe
- `ReclamacaoServiceTest.cls:48` — monta Case manualmente antes de chamar `ReclamacaoService.buildReclamacao`
- `CaseAreaParticipantePauseServiceTest.cls:18-38` — cadeia GestaoSLA/Categorizacao/RegrasSLA/AreaParticipante toda manual
- `AreaParticipanteSLABatchTest.cls:23,43,62` — mesma cadeia, 3 vezes

Estrutura proposta para a TestDataFactory central

Só recomendação — nada implementado. Ordem pensada para eliminar a maior duplicação primeiro:

1 createAccount() / createAccountWithContact()

Maior gap isolado — 19 + 13 classes recriam isso hoje

2 createCase(unidade, recordType)

Com tratamento automático embutido para a regra de Rebocadores (Navio/Porto) — elimina a duplicação tripla

3 createGestaoSLA(unidade) / createRegraSLA(...)

Já existem como helpers privados em `AreaParticipanteTestDataFactory` — só precisam virar públicos e parametrizáveis

4 createCategorizacao(...)

Unificar as 3 versões divergentes hoje espalhadas em 3 factories diferentes

5 createQueue() / createTestUser(profile)

12 + 8 classes montam isso na mão

6 createAreaParticipante(...)

Consolidar a factory já existente, hoje pouco reaproveitada fora do próprio domínio

7 createReclamacao(...) / createAgendamento(...)

Gaps reais — zero cobertura de factory hoje, mesmo com testes existentes desses domínios

8 Utilitários transversais — `pickValue`, `anyRecordType`, `anyOpenStatus` / `anyClosedStatus`

4 implementações quase idênticas hoje, uma em cada factory

03 Criticidades no código de produção

Revisão focada em Services, Selectors, Helpers, Controllers e Triggers do módulo de Atendimento (não testes; classes de Sales Cloud como cotação/lead ficaram fora de escopo). Os 2 achados críticos são de manutenibilidade/wiring, não vulnerabilidade explorável diretamente — os riscos de segurança encontrados são reais, porém mitigados por contexto (o dado não vem de input livre do usuário).

Crítico

Crítico · Arquitetura God Object: 4 domínios de negócio numa única classe

`GestaoSLAService.cls` – 1489 linhas, 28 métodos públicos

CRUD completo de `GestaoSLA__c` (782-936), `Categorizacao__c` (522-728), `MarcoSLA__c` (139-306) e `RegrasSLACategorizacao__c` (342-520), mais gate de permissões (1023-1060), resolução de escopo por Unidade de Negócio (1001-1021), validações de duplicidade/picklist (1310-1401) e mapeamento de DTO — tudo num único arquivo.

Por quê: qualquer mudança em uma das 5 responsabilidades força reteste/recompilação da classe inteira; alterar a distribuição de Categoria pode afetar cobertura de teste de Marco sem nenhuma relação lógica entre os dois. Risco de regressão cruzada alto e crescente a cada novo pacote — já é o maior arquivo de serviço do projeto.

Crítico · Wiring Funcionalidade implementada e testada, nunca conectada ao fluxo real

`CaseEntitlementAssignmentService.cls:6`

`assignForSalvador` (e a classe inteira) só é referenciada pelo próprio teste (`CaseEntitlementAssignmentServiceTest.cls`) — nenhum trigger handler, service ou controller de produção a invoca.

Por quê: ou é código morto que confunde manutenção futura (parece que a atribuição automática de Entitlement para Salvador está ativa, mas não está), ou é um bug real de wiring — atribuição automática de `EntitlementId` para Cases de Salvador silenciosamente nunca acontece por esse caminho. Precisa de uma decisão explícita: religar ou remover.

Alto

Alto · Arquitetura SOQL inline no trigger handler, com hack de teste para compensar

`CaseTriggerHandler.cls:35-44`

A query de bloqueio de fechamento de Case (`AggregateResult` contando `AreaParticipante__c` abertas) está inline no handler, apesar de existir `AreaParticipanteSelector.cls` no projeto — quebra o padrão "trigger fino" que o resto do handler segue.

Por quê: como a query não está isolada num Selector com método próprio, a única forma de simular resultado em teste é o hack `injectedBlockersByCase` (linhas 9, 32-34) — um seam de teste frágil que não seria necessário com a query no lugar certo.

Alto · Hardcoding

Nomes de Unidade de Negócio fixos em Apex, cobertura assimétrica

`CaseCreationSelector.cls:128-148` (`getBusinessUnitAliases`)

Trata "salvador" e "riogrande" com listas de alias fixas no código; Centro Logístico e Rebocadores não têm alias nenhum — apesar de `ParametrosAtendimento__mdt` existir no projeto exatamente para isso (já usado por `GestaoSLAHelper` / `ReclamacaoService` / `AtendimentoConfigService`).

Por quê: adicionar ou renomear uma unidade de negócio exige alterar Apex e fazer novo deploy em vez de só cadastrar um registro de Custom Metadata; a assimetria entre unidades é sintoma de a lógica ter sido remendada ao longo do tempo em vez de centralizada desde o início.

Médio

Médio · Arquitetura

SOQL inline disperso, sem Selector correspondente

`WSWillContextResolverService.cls` – linhas 200, 210-214, 236-241, 270-274, 298-304

5 queries inline na classe de serviço (incluindo SOQL dinâmico via `Database.queryWithBinds`), enquanto outras áreas do projeto (Case Creation, Categorização) já adotam o padrão Selector.

Por quê: inconsistência de camada aumenta custo de manutenção e dificulta reuso/mocking — a resolução de contato por telefone, por exemplo, é lógica que poderia ser reaproveitada por outros fluxos se estivesse num Selector.

Médio · Hardcoding

Valores de negócio duplicados em CMDT + fallback Apex

`CategorizacaoSelector.cls:56-63`

Fallback correto para quando `AtendimentoConfigService.getActiveConfigs()` retorna vazio, mas os valores de Salvador/Rio Grande/Centro Logístico/Rebocadores ficam duplicados em dois lugares: Custom Metadata e Apex.

Por quê: os dois lugares podem divergir silenciosamente se alguém atualizar o Custom Metadata sem atualizar o fallback (ou vice-versa) — risco de comportamento inconsistente entre ambientes onde o CMDT está populado e onde não está.

Médio · Tratamento de erro Falha "best-effort" só registrada em debug, sem monitoramento

`AreaParticipanteRegiaoAtendimentoService.cls:67-70`

Catch documentado e intencional ("uma falha aqui nunca deve impedir o salvamento do Case"), mas usa só `System.debug`.

Por quê: se a automação de Região de Atendimento Secundária falhar sistematicamente (ex.: erro de configuração em `GestaoSLA__c.RegiaoAtendimentoSecundaria__c`), ninguém é alertado — o log só existe numa sessão de debug ativa.

Médio · Tratamento de erro Padrão "engolir e retornar fallback" sem log em Helpers

`CaseRecategorizationHelper.cls:44-46` · `CaseClosureSurveyHelper.cls:84-86`

Ambos capturam qualquer exceção e retornam um valor de fallback sem nenhum registro, nem `System.debug` — o mesmo padrão aparece em `CaseClosureSurveyHelper.hasSurveyInvitationTodayByEmail` (linhas 65-87), onde a resolução dinâmica do campo de e-mail via `describe` falha silenciosamente para `false`.

Por quê: se o fallback estiver mascarando um bug de configuração (ex.: campo picklist ou `describe` inválido), o problema nunca aparece em nenhum monitoramento — um erro de configuração vira, sem log, uma decisão de negócio ("cliente não recebeu pesquisa hoje") tomada sobre dado errado.

Alto · Segurança SOQL dinâmico sem enforcement de FLS `AreaParticipanteSelector.cls:25-66`

`getAreasByCase`, `getAreaById`, `getAreasByIds` montam SOQL via `Database.query(soql)` sem `WITH USER_MODE / stripInaccessible`. O Service chamador só valida acesso a nível de objeto, não por campo.

Por quê: a lista retornada inclui campos sensíveis (`SolucaoTomada__c`, `ComentarioRetorno__c`) expostos a controllers `@AuraEnabled` — um usuário sem FLS nesses campos ainda os recebe, quebrando o modelo de segurança declarativo do Salesforce.

Alto · Segurança Query dinâmica de Contact sem FLS, dados de PII

`WSWillCaseCreationService.cls:614-682`

`queryContactsByIds` / `queryContactsByEmails` validam o campo extra contra o schema (mitiga injection), mas não aplicam enforcement de FLS na leitura de `Contact`.

Por quê: serve a integração WhatsApp/WSWill — um contexto de execução com sharing indevido pode expor e-mail/telefone além do que o usuário autenticado deveria ver.

Alto · Segurança Suppress de PMD sem justificativa documentada `ReclamacaoService.cls:60,134`

`@SuppressWarnings('PMD.ApexCRUDViolation')` em DML/SOQL de contexto de trigger, sem comentário explicando a decisão (diferente do padrão usado em `CaseOpeningNotificationService.cls:8`).

Por quê: padrão system-mode é aceitável aqui, mas a falta de justificativa documentada dificulta auditoria futura sobre se o suppress continua válido quando o fluxo mudar.

Alto · Segurança Suppress de PMD aplicado à classe inteira `CaseOpeningNotificationService.cls:8`

`@SuppressWarnings` no header da classe, não no método específico já revisado (linha 32-37).

Por quê: silencia o linter para todo código futuro adicionado à classe, não só para o SOQL system-mode já validado — uma nova query com FLS relevante passaria despercebida.

Médio (continuação)

Médio SOQL inline fora do Selector `AreaParticipanteService.cls:522`

`getParticipationDetails` executa `[SELECT IsStopped FROM Case ...]` direto, quebrando a convenção do próprio projeto de centralizar queries no Selector.

Por quê: duplicar o padrão fora do Selector dificulta auditoria centralizada de segurança/performance e abre precedente para mais SOQL inline se disseminado.

Médio · Arquitetura Selector com mesma concentração de domínios do Service

GestaoSLASelector.cls — 564 linhas, ~20 métodos

Espelha o achado do GestaoSLAService (seção Crítico): queries para 4 objetos diferentes num único arquivo.

Por quê: manutenção mais cara para localizar a query correta; caberia dividir por domínio (MarcoSLASelector, RegraSLACategorizacaoSelector) seguindo o padrão que outras partes do projeto já usam.

Baixo / referência positiva

- Baixo** WS_EmailToCaseInboundService.cls:158-160 — catch vazio, mas é o único que documenta explicitamente a decisão de engolir a exceção; modelo a seguir para os demais catches silenciosos listados acima.
- Referência** CaseTriggerHandler.cls — 78 linhas, delega quase tudo a Services especializados sem lógica duplicada (só a query de bloqueio, achado Alto acima, foge do padrão). Estrutura geral é o modelo que GestaoSLAService poderia seguir.
- Referência** Nenhum Selector órfão encontrado nos 9 arquivos *Selector.cls — achado histórico de auditorias anteriores já resolvido.

Bulkificação: nenhum SOQL/DML dentro de loop foi encontrado nas classes centrais analisadas — o padrão parece bem tratado no projeto. **Hardcoding por unidade de negócio:** a maior parte da resolução já passa por ParametrosAtendimento__mdt / AtendimentoConfigService, mas esse padrão não foi adotado de forma consistente — CaseCreationSelector e CategorizacaoSelector ainda têm nomes de unidade fixos em Apex (achados Alto e Médio acima), e ParametrosAtendimento__mdt é usado hoje em só 3 classes do projeto.

Análise estática, somente leitura — nenhum arquivo foi alterado. Escopo: force-app/main/default/classes, módulo de Atendimento/Service Cloud — classes de Sales Cloud (cotação, lead, entitlement de vendas) foram excluídas por não pertencerem a este domínio. Achados de segurança de FLS dinâmico assumem contexto @AuraEnabled/integração externa; não foram testados em runtime.